BÜYÜK VERİ ANALİZİNE GİRİŞ DÖNEM PROJESİ

1. GİRİŞ

Günümüzde kentsel mobilite verileri, şehir planlaması ve ulaşım stratejilerinin geliştirilmesinde kritik bir rol oynamaktadır. New York City (NYC) gibi metropollerde taksi hareketliliği, şehrin ekonomik ve sosyal nabzını tutan devasa bir veri hacmi oluşturmaktadır. Bu proje kapsamında, **NYC Yellow Taxi** verileri kullanılarak ham verinin sisteme girişinden, işlenmesine, depolanmasına ve nihayetinde bir makine öğrenmesi modeli ile anlamlı tahminler üretmesine kadar olan süreçleri kapsayan **uçtan uca (end-to-end)** bir **büyük veri boru hattı (pipeline)** tasarlanmıştır.

Projenin temel motivasyonu, yüksek hacimli ve sürekli akan verilerin ölçeklenebilir bir mimari üzerinde, modern veri mühendisliği araçlarıyla nasıl yönetilebileceğini simüle etmektir. Bu bağlamda; veri akışı, gerçek zamanlı işleme, **ACID uyumlu depolama ve model takibi** gibi endüstriyel standartlardaki bileşenler bir araya getirilmiştir.

2. PROJE MİMARİSİ VE TEKNİK ALTYAPI

Bu projenin mimarisi, modern veri mühendisliği ekosisteminde kabul görmüş **"Containerized Microservices"** ve **"Medallion Architecture"** (Katmanlı Veri Gölü) prensipleri üzerine inşa edilmiştir. Sistemin tüm bileşenleri Docker ekosistemi içerisinde izole edilmiş servisler olarak kurgulanmış olup, her bir servis önceden tanımlanmış "Docker Network" köprüleri üzerinden birbiriyle asenkron ve güvenli bir şekilde haberleşmektedir.

2.1. Altyapı ve Konteynerizasyon (Docker & Docker Compose)

Sistemin taşınabilirliğini ve tekrarlanabilirliğini sağlamak amacıyla tüm ekosistem Docker Compose ile yapılandırılmıştır.

- **Spark-Env:** PySpark, Delta Lake bağımlılıkları ve Jupyter Lab arayüzünü barındıran özelleştirilmiş bir imajdır.
- **Kafka-Producer:** Hafif bir Python imajı olup, ham veri setini Kafka topic'lerine akıtma görevini üstlenir.
- **Servis Orkestrasyonu:** Kafka, Zookeeper, Spark Master/Worker, MLflow ve UI arayüzleri tek bir komutla ayağa kaldırılacak şekilde konfigüre edilmiştir.

2.2. Veri Akışı ve Mesajlaşma Katmanı (Apache Kafka)

Veri akışının gerçek zamanlı simülasyonu için Apache Kafka kullanılmıştır.

- **Veri Üretimi:** Yerel diskte bulunan Parquet formatındaki NYC Taxi verileri, Python tabanlı bir producer tarafından okunarak JSON formatına dönüştürülür.
- **Topic Yapısı:** Veriler, ölçeklenebilirlik göz önünde bulundurularak ilgili topic'lere asenkron bir şekilde gönderilir. Bu katman, veri işleme birimi (Spark) ile veri kaynağı arasındaki bağımlılığı (decoupling) ortadan kaldırır.

2.3. Veri İşleme ve Depolama (Spark Streaming & Delta Lake)

Projenin en kritik bileşeni olan bu katman, verinin ham halden işlenmiş bilgiye dönüşümünü yönetir.

- **Structured Streaming:** Apache Spark, Kafka topic'lerini sürekli dinleyerek gelen mikro-batch'leri işler.
- **Delta Lake (Medallion Architecture):** Veri ambarı yönetimi için üç aşamalı bir yapı kurgulanmıştır:
    1. **Bronze (Raw):** Kafka'dan gelen ham verinin hiçbir değişikliğe uğramadan kaydedildiği katman.
    2. **Silver (Cleaned):** Şema doğrulaması yapılmış, veri tipleri düzeltilmiş ve eksik verilerden arındırılmış katman.
    3. **Gold (Aggregated/Feature):** ML modelleri için optimize edilmiş, öznitelik mühendisliği (Feature Engineering) uygulanmış nihai katman.

2.4. Makine Öğrenmesi ve Deney Takibi (Spark MLlib & MLflow)

Veri boru hattının sonunda, Gold katmanındaki veriler kullanılarak makine öğrenmesi süreçleri işletilir.

- **ML Pipeline:** Özelliklerin vektörleştirilmesi ve model eğitimi adımları Spark MLlib üzerinden gerçekleştirilir.
- **MLflow Entegrasyonu:** Eğitilen 5 farklı modelin hiperparametreleri, eğitim süreleri ve başarı metrikleri MLflow sunucusuna kaydedilir. Bu sayede model versiyonlama ve karşılaştırma süreçleri sistematik bir yapıya kavuşturulmuştur.

3. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLER

3.1. Jupyter Notebook Arayüzü ve Backend Senkronizasyon Sorunları

Büyük veri işleme süreçlerinde interaktif bir arayüz sunan Jupyter Lab kullanımı sırasında bazı performans ve izleme (monitoring) sorunları gözlemlenmiştir:

- **Arayüz Gecikmeleri:** Karmaşık Spark job'ları ve streaming işlemleri sırasında Jupyter arayüzünde yavaşlamalar meydana gelmiştir. Bu durumun, Docker konteynerine ayrılan bellek limitlerinden ve tarayıcı tabanlı arayüzün yoğun veri loglarını işleme kapasitesinden kaynaklandığı tespit edilmiştir.
- **İşlem Durumu Belirsizliği:** Özellikle Spark Structured Streaming hücreleri çalışırken, arayüzün backend tarafındaki işlemin devam edip etmediğine dair anlık geri bildirim vermediği (takılma hissi) fark edilmiştir. Kullanıcı, işlemin hala aktif mi olduğu yoksa bir hata sonucu durup durmadığı konusunda belirsizlik yaşamıştır.
- **Çözüm:** Bu sorunu aşmak için Spark'ın kendi izleme arayüzü olan **Spark UI (localhost:4040)** aktif olarak kullanılmıştır. Notebook arayüzündeki belirsizlik, Spark UI üzerinden job progress, executor durumu ve DAG (Directed Acyclic Graph) takibi yapılarak çözüme kavuşturulmuştur.

4. SONUÇLAR VE DEĞERLENDİRME

Proje kapsamında kurulan uçtan uca büyük veri pipeline'ı başarıyla çalıştırılmış ve NYC Yellow Taxi verileri üzerinden anlamlı tahminleme sonuçları elde edilmiştir. Bu bölümde, MLflow ile takip edilen model performansları ve Streamlit dashboardu üzerinden elde edilen çıkarımlar özetlenmiştir.

4.1. Model Performans Karşılaştırması

Regresyon problemi çerçevesinde eğitilen 5 farklı modelin (Decision Tree,Random Forest, GBT,Linear Regression ve GLR) performansları RMSE ve R^{2} metrikleri baz alınarak karşılaştırılmıştır.

- **En İyi Performans:** Yapılan deneyler sonucunda, karmaşık veri yapılarını ve doğrusal olmayan ilişkileri daha iyi yakalayabilen **Random Forest** modelinin en düşük hata payı (RMSE) ile en yüksek başarıyı sergilediği gözlemlenmiştir. Random Forest Modeli başarı sonuçları: RMSE:5.537800 , MAE:2.777700 , R^2:0.922100
- **GBT Regressor Modeli** başarı sonuçları: RMSE:5.796500 , MAE:2.911100 , R^2:0.914700
- **GLR Modeli** başarı sonuçları: RMSE:6.426800 , MAE:3.423700 , R^2:0.895100
- **Linear Regression Modeli** başarı sonuçları: RMSE:6.426800 , MAE:3.423700 , R^2:0.895100
- **Decision Tree Modeli** başarı sonuçları: RMSE:6.156500 , MAE:3.373000 , R^2:0.903800
- **MLflow Entegrasyonu:** Tüm model parametreleri ve metrikleri MLflow arayüzü üzerinden kayıt altına alınmış; bu sayede modeller arasındaki performans farkları objektif verilerle analiz edilmiştir.

4.2. Öznitelik Önem Düzeyi (Feature Importance)

Modelin tahminleme başarısı, girdi olarak verilen özelliklerin (features) hedef değişken (total_amount) üzerindeki ağırlıklarına dayanmaktadır. Yapılan analizler sonucunda özniteliklerin etki sıralaması şu şekildedir:

1. Yolculuk Mesafesi (trip_distance): Toplam ücretin ana belirleyicisidir. NYC Taksi sisteminde temel ücretlendirme kat edilen mesafe üzerinden yapıldığı için model bu değişkeni birincil ağırlık olarak kullanır.

2. Yolculuk Süresi (trip_duration_min): Trafik yoğunluğunu ve bekleme maliyetini temsil eder. Özellikle kısa mesafeli ancak trafiğin yoğun olduğu bölgelerdeki ücret artışlarını modelin öğrenmesini sağlar.

3. Ortalama Hız (average_speed): Yolculuk esnasındaki trafik akışkanlığını ifade eder. Düşük hızlar yoğun trafiği, yüksek hızlar ise akıcı yolu işaret eder. Model, hız düştükçe ücretin (zaman bazlı artış nedeniyle) yükseleceğini bu değişken üzerinden doğrular.

4. Zaman ve Trafik Faktörleri (pickup_hour , is_rush_hour , day_of_week)

Saat Etkisi: Gece/gündüz tarifesi ve yoğun saat (rush hour) ek ücretlerinin (surcharge) tahminlenmesini sağlar.

Haftalık Döngü: Hafta içi iş trafiği ile hafta sonu eğlence/gezi trafiği arasındaki karakter farkını yansıtır.

5. Lokasyon ve İstisnalar (is_airport_trip): Havaalanı (JFK, Newark vb.) yolculukları genellikle sabit tarifeye (Flat Rate) tabidir. Model, bu ikili (binary) özellik sayesinde normal mesafe hesabına uymayan bu özel durumları yakalar.

4.3. Dashboard ve Görsel Analiz Çıktıları

Projenin görselleştirme ve kullanıcı arayüzü aşamasında, Python tabanlı modern bir web framework'ü olan **Streamlit** teknolojisi kullanılmıştır. Dashboard; veri analitiği sonuçlarını, model performans metriklerini ve öznitelik önem düzeylerini etkileşimli grafiklerle sunmaktadır. Pandas ile işlenen veriler; Seaborn ve Matplotlib kütüphaneleri aracılığıyla görselleştirilerek; NYC taksi ücret dağılımları, yolcu sayıları ve saatlik trendler gibi kritik içgörüler kullanıcı dostu bir arayüzde birleştirilmiştir. Temel amaç, karmaşık makine öğrenmesi sonuçlarını **(MAE, RMSE, R^2 gibi değerler)** ve modelin karar mekanizmasını teknik olmayan paydaşlar için anlaşılır ve izlenebilir bir yapıya dönüştürmektir.

4.4. Genel Değerlendirme

Bu çalışma; Docker, Kafka, Spark ve Delta Lake gibi teknolojilerin entegre bir şekilde kullanılabileceğini kanıtlamıştır. Mimari, özellikle **"Medallion Architecture"** sayesinde veri kalitesini her aşamada korumuş ve makine öğrenmesi modelleri için temiz bir veri kaynağı (Gold Table) sunmuştur. Sistemin modüler yapısı, gelecekte daha büyük veri setleri veya farklı veri kaynakları eklendiğinde kolayca ölçeklenebileceğini göstermektedir.

Melis Portakal 220201035
Batın Erke Öztürk 220201031
Halil Han Kavak 220201065
Gülsüm Nur Maslak 220201033